SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I A. Midhuna (192521302) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

A. Midhuna
Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

ANSWER:

Scenario: Debugging Hash Function Implementation (SHA Family)

A. Identify errors in message padding and block processing steps

Scenario: A company uses a Python program based on the SHA family to generate hash values for files. The program produces inconsistent hash outputs for the same input, causing problems when verifying file integrity.

Errors and solutions:

1. Incorrect padding: SHA algorithms require padding with a 1 bit, followed by 0 bits and the original message length. Incorrect padding produces invalid hash values.
2. Block processing errors: The input must be divided into fixed-size blocks, such as 512-bit blocks for SHA-256.
3. Incorrect message length: The original message length must be appended in the format required by the selected SHA algorithm.
4. Wrong initialization: The algorithm must use the correct initial hash values and round constants.

B. Fix issues related to bitwise operations and endianness handling

1. Bitwise operations: Use the correct XOR (^), AND (&), OR (|), and right-shift (>>) operations.

2. 32-bit arithmetic: Apply a mask such as 0xFFFFFFFF to keep intermediate values within 32 bits when implementing SHA-256 manually.
3. Endianness: SHA-256 interprets message words and encodes the final message length in big-endian format.
4. Rotation operations: Implement right rotations correctly to prevent incorrect hash calculations.
5. Testing: Compare the program's output against Python's trusted hashlib.sha 256() implementation using standard test inputs.

C. Optimize processing speed for large files (50 MB or more)

1. Use Python's built-in hashlib module instead of implementing SHA-256 manually.
2. Read files in chunks rather than loading the entire file into memory.
3. Update the hash incrementally using the update () method.
4. Use a suitable chunk size, such as 1 MB, to balance memory usage and processing efficiency.

Example Python code:

```
import hashlib
def file_hash(filename):
    h = hashlib.sha 256()
    with open (filename, "rb") as f:
        while True:
            data = f.read(1024 * 1024)
            if not data:
                break
            h.update(data)
    return h.hexdigest()
print(file_hash("largefile.bin"))
```

D. Evaluate secure hash functions compared to basic checksums

Secure hash functions (SHA-256)	Basic checksums
Designed to resist deliberate attacks	Mainly detect accidental errors
Very difficult to find two inputs with the same hash	Collisions are generally easier to produce
Suitable for security and integrity verification	Suitable for simple error detection
Used in digital signatures and file verification	Used in basic file-transfer checks

Conclusion

SHA-256 provides strong integrity verification by generating a fixed-length hash for input data. Correct padding, block processing, bitwise operations, and endianness are essential for consistent results. For files of 50 MB or more, Python's built-in hashlib with chunk-based reading provides efficient and reliable processing. Secure hashes are preferable to basic checksums when protection against intentional tampering is required.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation